

ResumeMatch Lab — A Methodology Case Study

Designing a rigorous A/B test for resume–job match quality

ResumeMatch Lab

2026

1. Executive Summary

ResumeMatch Lab answers a deceptively simple question a job seeker actually has: *“I have two versions of my resume — which one matches more jobs, by how much, and where?”* We treat it as a designed experiment rather than a vibe check. Both resume variants are embedded with a sentence-transformer (BAAI/bge-small-en-v1.5, 384-dim) and scored by cosine similarity against a fixed snapshot of **2,000 real Indian tech job postings**. Because every job is scored by **both** resumes, the data is naturally **paired**, and the per-job score difference $d_i = \text{score_B}(i) - \text{score_A}(i)$ becomes the unit of analysis.

On that paired-delta vector we run a deliberately over-complete battery of methods, each chosen to answer a different question:

Question	Method
Is the average difference real?	Paired t-test / Wilcoxon (normality-gated)
How big, with what uncertainty?	Cohen’s d ; bootstrap BCa 95% CI
Could we even have detected it?	Power analysis; achieved power; MDE table
Can we tighten the estimate?	CUPED variance reduction
Is it valid if we peek early?	mSPRT always-valid p-value
How probable is “B is better”?	Bayesian Beta-Binomial posterior
Where does the effect live?	Per-cluster tests + Bonferroni / BH-FDR

The key methodological choices — a paired design, a normality gate for test selection, BCa intervals, job-side CUPED covariates, a mixture sequential test, and multiplicity control across 8 job clusters — are explained and justified in the sections that follow. A reader who only has 90 seconds should read this section and Section 12 (the decision framework).

2. Problem Framing

A job seeker rarely has one resume; they have variants — a “data” version and a “platform” version, a keyword-dense version and a narrative version. The decision they want to make is concrete: **which variant to send**, and whether the choice even matters. The naive approach (generic ATS keyword scoring) ignores the job market the resume is actually competing in, and reports a single number with no uncertainty.

We reframe the question as an experiment with a clear estimand: the mean difference in match score between variant B and variant A across a representative population of jobs. This buys us everything experimental design offers — effect sizes, confidence intervals, power, and the ability to say *“the lift is concentrated in ML/AI roles; A still wins for Backend.”*

3. Hypothesis and Metric Design

Hypotheses. For mean paired difference `mu_d`:

$$H_0 : \mu_d = 0 \quad H_1 : \mu_d \neq 0$$

Primary metric. The per-job paired delta and its mean:

$$d_i = \text{score}_B(i) - \text{score}_A(i), \quad \hat{d} = \frac{1}{N} \sum_{i=1}^N d_i$$

A **paired** design is the right model because the same 2,000 jobs are scored by both resumes; pairing removes job-to-job difficulty as a nuisance source of variance and is strictly more powerful than a two-sample comparison here.

Guardrail metrics (surfaced to the user, not used to declare victory):

- *Length parity.* If $|\text{len}(A) - \text{len}(B)| / \max(\text{len}(A), \text{len}(B)) > 0.5$, warn — wildly different lengths can bias embeddings.
- *Skill-set parity.* If the Jaccard similarity of extracted skill sets < 0.3 , warn — the two documents may be different roles, not A/B variants of one.
- *Parse-quality floor.* If either resume parses to < 200 characters, **refuse** to score rather than report nonsense.

4. Embedding-Based Scoring

We embed text with `BAAI/bge-small-en-v1.5`, a 384-dimensional sentence-transformer, and measure match by cosine similarity. Three deliberate choices matter:

1. **Model reuse / consistency.** The same model and preprocessing embed the job corpus (inherited from the sibling project *JobAtlas*) and the resumes. Identical encoder, no instruction prefix, `normalize_embeddings=True` on both sides. If the resume were embedded differently from the jobs, cosine similarity would be meaningless.
2. **Normalized vectors.** Because both sides are L2-normalized, cosine similarity reduces to a dot product, so scoring 2 resumes against 2,000 jobs is a single $(2000 \times 384) \cdot (384,)$ matrix-vector product — milliseconds on CPU.
3. **Cosine as a proxy.** Cosine similarity is a proxy for *textual* relevance, not for hiring outcomes. We state this limitation plainly (Section 13) rather than overclaim.

5. Job Clusters and Stratification

A single global verdict hides the most useful insight: a resume can win overall yet lose in the roles the candidate actually targets. We therefore cluster the 2,000 job embeddings with **K-means (k=8)** and label clusters from their dominant titles: *Data Engineering*, *Data & Analytics*, *Machine Learning / AI*, *DevOps / SRE / Cloud*, *Backend Engineering*, *Frontend / Mobile*, *Product Management*, *QA / Testing*. Clusters are computed once with a fixed seed and committed alongside the snapshot for reproducibility.

Alternatives considered: LDA topic modeling (softer assignments, harder to label crisply) and rule-based title matching (brittle to title variety). K-means on embeddings gave stable, interpretable groups and feeds directly into the stratified analysis of Section 11.

6. Frequentist Inference

Test selection is data-driven. We run a Shapiro-Wilk normality check on the deltas (subsampling at 5,000 where N is larger, since the statistic saturates). If the deltas are approximately normal we use the **paired t-test**; otherwise the **Wilcoxon signed-rank** test, which is distribution-free and robust to the heavy tails embedding deltas sometimes exhibit.

Paired t-test statistic and parametric CI:

$$t = \frac{\hat{d}}{s_d/\sqrt{N}}, \quad \text{df} = N - 1, \quad \hat{d} \pm t_{0.975, N-1} \frac{s_d}{\sqrt{N}}$$

Bootstrap CIs. Independent of the parametric assumption, we resample the paired-delta vector **10,000** times and report both the percentile interval and the **bias-corrected and accelerated (BCa)** interval. BCa corrects for skew and median bias via a bias-correction term $\mathbf{z}_0$ and a jackknife acceleration $\mathbf{a}$, and is our headline interval because the percentile interval can be optimistic under skew.

7. Power Analysis

With $N = 2,000$ paired observations, the design is highly powered. We report three things:

- **Required N** for 80% power at the observed effect size (often well under 100 — i.e., 2,000 jobs is comfortably enough).
- **Achieved power** at the observed effect. For large noncentrality, SciPy’s noncentral-t saturates to NaN; we fall back to the large-sample normal approximation `power ~ Phi(|d|sqrt N - z_{1-alpha/2})`, which is effectively exact at this N .
- A **minimum detectable effect (MDE)** table in Cohen’s d across an alpha x power grid, so a reader can see exactly how small an effect this design could have caught.

Reporting achieved power matters most when a result is *non-significant*: it lets us say “we could detect $d \geq 0.06$ at 80% power; the observed d was 0.03,” distinguishing “no effect” from “underpowered.”

8. CUPED Variance Reduction

CUPED (Controlled-experiment Using Pre-Experiment Data; Deng et al., 2013) reduces the variance of the effect estimate by regressing out variation explained by covariates that are independent of the treatment contrast:

$$\hat{d}_i^{\text{adj}} = d_i - X_i^\top \hat{\beta} + \bar{d}, \quad \text{variance reduction} = 1 - \frac{\text{var}(\hat{d}^{\text{adj}})}{\text{var}(d)} = R^2$$

A correction to the naive covariate choice. The original project spec named *resume-level* covariates (resume length, skill density). These are **constant across the 2,000 jobs**, so they cannot reduce per-job delta variance — including them does nothing. The unit of observation is a *job*, so the covariates must vary across jobs and be independent of the A-vs-B contrast. We therefore use **job-side covariates**: cluster membership (one-hot) and job-description length. Cluster membership is the workhorse: if B systematically beats A in ML/AI but loses in Backend, cluster explains a large share of the delta variance, and residualizing on it tightens the interval. The achieved variance reduction equals the R^2 of this regression. We document this deviation openly because an interviewer *should* ask “why those covariates?” — and the honest, correct answer is stronger than a plausible-but-wrong one.

9. Sequential Testing (mSPRT)

A fixed-horizon p-value is only valid if you look once. The **mixture Sequential Probability Ratio Test** (Robbins, 1970) yields an **always-valid** p-value: you may peek at any sample size without inflating Type I error. With a normal mixture prior $N(0, \tau^2)$ on the alternative mean and per-job variance estimate σ^2 , the mixture likelihood ratio after n observations is

$$\Lambda_n = \sqrt{\frac{\sigma^2}{\sigma^2 + n\tau^2}} \exp\left(\frac{n^2 \tau^2 \bar{d}_n^2}{2\sigma^2(\sigma^2 + n\tau^2)}\right), \quad p_n = \min\left(1, \min_{m \leq n} \frac{1}{\Lambda_m}\right).$$

In the static-snapshot product all 2,000 jobs are scored at once, so mSPRT isn't an operational stopping rule here; we display it as a "valid at any sample size" alternative p-value and plot its trajectory. Its real value is conceptual honesty — and it is a genuine differentiator in a Product Analyst interview. We verify the implementation by simulation: across many null runs, the empirical "ever reject" rate stays at or below alpha.

10. Bayesian Framing

We complement the frequentist machinery with a conjugate Bayesian model that answers a question users find intuitive: *what is the probability B beats A on a randomly chosen job?* Treat each job as a Bernoulli trial $[d_i > 0]$, let k be the number of B-wins:

$$p \sim \text{Beta}(1, 1), \quad p \mid \text{data} \sim \text{Beta}(1 + k, 1 + N - k).$$

We report the posterior mean, a 95% credible interval, and $P(p > 0.5 \mid \text{data})$ — the probability B wins more often than not — plus the posterior density. The Bayesian posterior and the frequentist test rarely disagree at $N = 2,000$, but the posterior communicates *uncertainty as probability*, which is far more natural for a non-technical user than a p-value.

11. Multiple Comparisons

Running a paired test within each of the 8 clusters produces 8 p-values; uncorrected, the family-wise error rate inflates badly. We report two corrections:

- **Bonferroni** — conservative; controls FWER by testing each cluster at $\alpha/8 = 0.00625$.
- **Benjamini-Hochberg FDR** — less conservative; controls the expected false-discovery rate at 0.05, more appropriate when we expect several clusters to show real effects.

Each cluster is flagged as a win for B (green), a win for A (red), or not significant (grey) **after** correction, and rendered as a forest plot with 95% CI whiskers.

12. Decision Framework

The single recommendation combines three independent lines of evidence:

```
significant := (primary_p < 0.05) AND (bootstrap_BCa_CI excludes 0)
winner      := B if significant and mean_delta > 0
              A if significant and mean_delta < 0
              tie otherwise
confidence   := high      if primary_p < 0.01 and max(P(B>A), 1-P(B>A)) > 0.99
              moderate    if primary_p < 0.05 and ... > 0.95
              low          otherwise
```

Requiring **both** a significant test **and** a bootstrap interval that excludes zero guards against a single method’s fragility. Confidence is then graded by combining frequentist and Bayesian extremity, and the per-cluster table tells the user *where* to trust the headline.

Worked example (illustrative synthetic run). Comparing a DevOps-flavored Resume A against a Data-Science-flavored Resume B over the 2,000-job corpus produced: Resume A wins overall by **2.08 points** (BCa 95% CI [1.84, 2.30]). The normality gate rejected normality, so the engine auto-selected the **Wilcoxon** test ($p \sim 7e-75$); Cohen’s $d = -0.40$, achieved power ~ 1.00 . CUPED cut variance by **54.9%** (effective N $\times 2.22$), driven by between-cluster structure. mSPRT’s always-valid $p \sim 3e-67$ (reject). The Bayesian posterior put $P(B > A \text{ per job}) \sim 0.00$. Crucially, the **per-cluster** view inverted the headline where it counts: **B won Machine Learning / AI (+3.87 pts)** while A dominated Data Engineering, DevOps, and Backend. That nuance — “send B for ML roles, A for infra roles” — is the entire point of the product.

13. Limitations and Future Work

- **Embedding-as-proxy.** Cosine similarity measures textual relevance, not recruiter behavior or hiring outcomes. A higher score is evidence of better keyword/semantic overlap, not a guaranteed callback.
- **Snapshot recency.** The corpus is a fixed 2,000-job snapshot; the labor market drifts. A scheduled refresh (inherited from JobAtlas) mitigates but does not eliminate staleness.
- **No recruiter-side validation.** We never observe whether higher-scoring resumes actually get more interviews — the ground truth we’d need to validate the proxy.
- **CUPED covariates** are job-side and modest; a richer covariate set (job seniority, salary band) could reduce variance further.
- **Future work:** calibrate scores against real callback data; per-role embeddings; counterfactual “what one edit would most raise your ML/AI score” guidance.

14. References

- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed.
- Robbins, H. (1970). “Statistical methods related to the law of the iterated logarithm.” *Annals of Mathematical Statistics*, 41(5). (mixture SPRT / always-valid inference)
- Deng, A., Xu, Y., Kohavi, R., Walker, T. (2013). “Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data (CUPED).” *WSDM*.
- Benjamini, Y., Hochberg, Y. (1995). “Controlling the False Discovery Rate.” *JRSS-B*, 57(1).
- Efron, B., Tibshirani, R. (1993). *An Introduction to the Bootstrap*. (BCa intervals)

This methodology document is licensed CC-BY-SA 4.0. The accompanying code is Apache-2.0.